CONCURRENCY CONTROL WITH TIMESTAMPING METHODS
&
DEADLOCK

Priya R

TIMESTAMP-BASED PROTOCOL

- Another method for determining the serializability order is to select an ordering among transactions.
- The most common method for doing so is to use a "Timestamp - ordering" scheme.

TIMESTAMPS

- With each transaction Ti in the system , we associate a unique fixed timestamp denoted bt TS(Ti) .
- This timestamp is assigned by the database before the transaction Ti starts execution.
- If a transaction Ti has been assigned timestamp TS(Ti) , and a new transaction Tj enters the system , then TS(Ti) < TS(Tj).

METHODS TO IMPLEMENT

There are two simple methods for implementing timestamp :

1. Use the value of the **System clock** as the timestamp :
   A transactions timestamp is equal to the value of the clock when the transaction enters the system.
2. Use a **Logical counter** that is incremented after a new timestamp has been assigned. A transactions timestamp is equal to the value of the counter when the transaction enters the system

- The timestamps of the transactions determine the serializability order.Thus, if TS(Ti) < TS(Tj) ,Then the system must ensure that the produced schedule is equivalent to a serial schedule in which transaction Ti appears before transaction Tj.
- To implement this scheme,we associate with each data item Q two timestamp values.
- **W-timestamp(Q) :** denotes the largest timestamp of any transaction that executed Write(Q) succesfully.
- **R-timestamp(Q) :** denotes the largest timestamp of any transaction that executed read(Q) successfully.
- These timestamps are updated whenever a new read(Q) or write(Q) instruction is executed.

THE TIMESTAMP-ORDERING PROTOCOL

- The timestamp-ordering protocol ensures that any conflicting read and write operations are executed in timestamp order.
- This protocol operates as follows:

1. Suppose that transaction Ti issues read(Q).

a. If TS(Ti)<W-timestamp(Q), then Ti needs to read a value of Q that was already overwritten. Hence, the read operation is rejected, and Ti is rolled back.

b. If TS(Ti)>=W-timestamp(Q), then read (Q) is executed and R-timestamp(Q) is set to the maximum of R-timestamp(Q) and TS(Ti).

2. Suppose that transaction *Ti* issues write(*Q*).

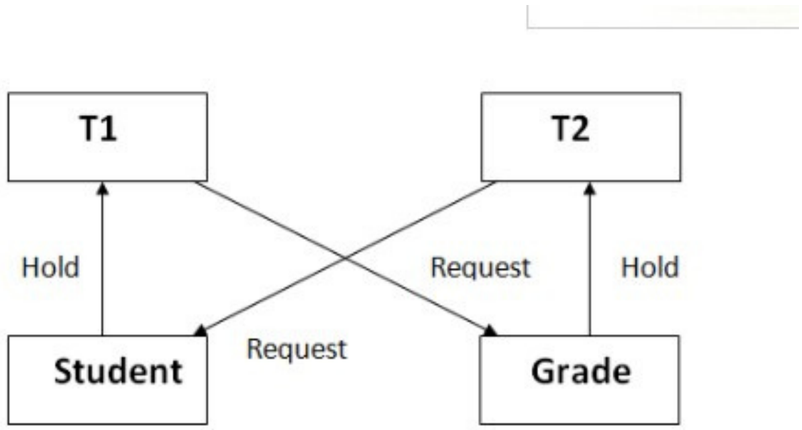
a. If TS(Ti)< R-timestamp(Q) then the value of Q that Ti is producing was needed previously, and the system assumed that that value would never be produced. Hence, the system rejects the write operation and rolls Ti back.

b. If TS(Ti) < W-timestamp(Q), then Ti is attempting to write an obsolete value of Q. Hence, the system rejects this write operation and rolls Ti back. c. Otherwise, the system executes the write operation and sets W-timestamp(Q) to TS(Ti).

- If a transaction Ti is rolled back by the concurrency-control scheme as result of issuance of either a read or write operation, the system assigns it a new timestamp and restarts it.
- The timestamp-ordering protocol ensures **conflict serializability.**
- The protocol ensures **freedom from deadlock,** since no transaction ever waits.

DEADLOCK

- In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for one another to give up locks.



DEADLOCK

- The figure shows two transactions T1 and T2 ,and 2 resources grade and student.
- T1 holds a lock on student table and wants to access the grade table,which is already locked by T2.
- T2 holds a lock on the grade table and is requesting access to the student table, which is locked by T1.
- This results in a circular wait : T1 is waiting for T2 to release grade and T2 is waiting for T1 to release student.
- Since both are waiting on each other and none can proceed ,the system is in a deadlock.

DEADLOCK HANDLING

1. Deadlock avoidance
2. Deadlock detection
3. Deadlock prevention

1. **Deadlock avoidance :**

Deadlock avoidance is a resource allocation strategy that ensures the system remains in a safe state by checking whether granting a resource request will potentially lead to a deadlock. If there is a risk, the request is delayed or denied.

DEADLOCK HANDLING

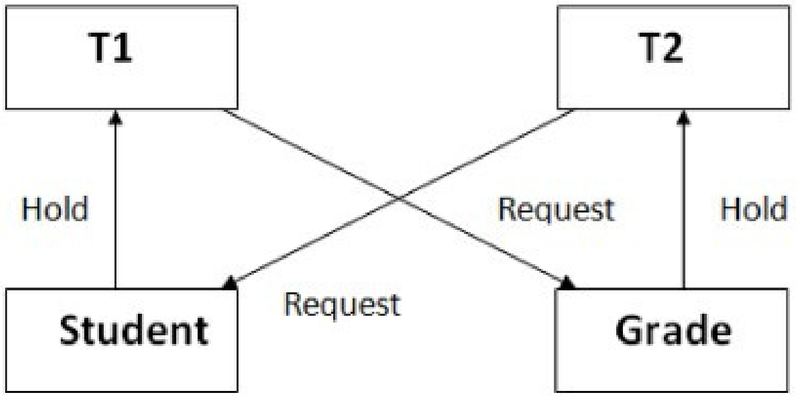
2.**Deadlock Detection :**

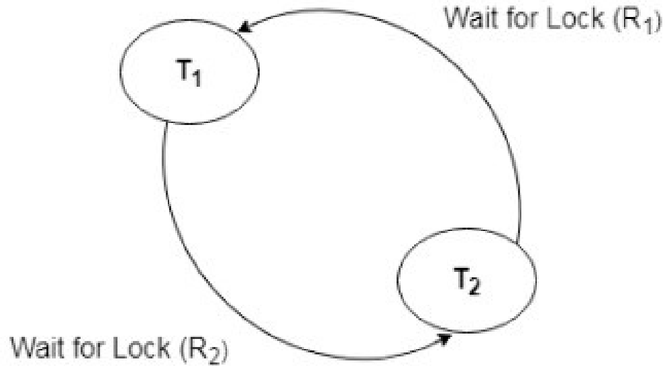
- When a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is involved in a deadlock or not.
- The lock manager maintains a **Wait for the graph** to detect the deadlock cycle in the database.

WAIT FOR GRAPH

- This is the suitable method for deadlock detection.
- In this method, a graph is created based on the transaction and their lock.
- If the created graph has a cycle or closed loop, then there is a deadlock.
- The wait for the graph is maintained by the system for every transaction which is waiting for some data held by the others.
- The system keeps checking the graph if there is any cycle in the graph.

WAIT FOR GRAPH





- Here the deadlock is detected because the graph is cyclic.

EXAMPLE FOR NO DEADLOCK GRAPH

- Consider T1 is holding a lock on resource A.
- T2 is waiting for T1 to release Resource A.
- T3 is waiting for T2 to release a resource B.
- T2 → T1 (T2 is waiting for T1)
- T3 → T2 (T3 is waiting for T2)
- Graph :

T3 → T2 → T1

- The graph is a straight line.it doesnt form a cycle.
- Since no cycle exists , no deadlock occurs.

DEADLOCK PREVENTION

- Deadlock prevention method is suitable for a large database.
- If the resources are allocated in such a way that deadlock never occurs, then the deadlock can be prevented.
- Two schemes for deadlock prevention:

1. Wait-Die scheme
2. Wound wait scheme

WAIT - DIE SCHEME

- In this scheme, If a transaction requests a resource that is locked by another transaction, then the DBMS simply checks the timestamp of both transactions and allows the older transaction to wait until the resource is available for execution.
- There are two transactions Ti and Tj and let TS(T) is a timestamp of any transaction T.
- If T2 holds a lock by some other transaction and T1 is requesting for resources held by T2 then the following actions are performed by DBMS:

WAIT - DIE SCHEME

- Check if TS(Ti) < TS(Tj) - If Ti is the older transaction and Tj has held some resource, then it allows Ti to wait until resource is available for execution. That means if a younger transaction has locked some resource and an older transaction is waiting for it, then an older transaction is allowed to wait for it till it is available .
- If T1 is an older transaction and has held some resource with it and if T2 is waiting for it, then T2 is killed and restarted later with random delay but with the same timestamp. i.e. if the older transaction has held some resource and the younger transaction waits for the resource, then the younger transaction is killed and restarted with a very minute delay with the same timestamp. This scheme allows the older transaction to wait but kills the younger one.

WOUND - WAIT SCHEME

- In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After the minute delay, the younger transaction is restarted but with the same timestamp.
- If the older transaction has held a resource which is requested by the Younger transaction, then the younger transaction is asked to wait until older releases it.

THANK YOU

Peak Time Learners